

Lab 3.4 — NeMo Guardrails: Política Conversacional Declarativa

Qué es este lab

Prueba el mismo criterio de bloqueo que la política Rego del Lab 3.1 (ninguna acción DELETE contra datos de producción), pero implementado como *input rail* declarativo (NVIDIA NeMo Guardrails) en vez de sidecar HTTP de OPA. El punto pedagógico es de capas, no de sustitución: OPA evalúa una acción ya decidida por el agente (autorización *post*-decisión -el LLM ya eligió invocar la tool, y ahí se lo frena); NeMo Guardrails intercepta el mensaje del usuario *antes* de que el LLM siquiera razone sobre él (filtrado *pre*-decisión). Un sistema robusto usa ambas.

Las piezas que corren

- **FakeListLLM**: un LLM de prueba determinista, que siempre devuelve una respuesta fija de una lista predefinida. No hace red, no tiene API key, no depende de ningún modelo real -sirve para verificar el *mecanismo* del rail (¿bloquea o no bloquea?) sin el ruido ni la latencia de un LLM de verdad.
- **check_dangerous_tool_request**: la acción Python determinista (sin LLM) que decide si un mensaje es peligroso -mismo criterio por substring que la política Rego del Lab 3.1 (`delete`, `eliminar`, `drop table`, etc.).
- El **input rail** (Colang): ejecuta la acción; si devuelve verdadero, corta el flujo y responde con el mensaje de rechazo -el mensaje del usuario nunca llega al LLM principal.
- **Modelo local**: el mismo rail puede apuntar a `qwen3.5:9b` vía Ollama en vez de a un modelo en la nube, usando el cliente OpenAI-compatible que expone Ollama.

El flujo

1. El usuario manda un mensaje.
2. El input rail lo intercepta y llama a `check_dangerous_tool_request`.
3. Si el resultado es verdadero (mensaje peligroso), el rail corta el flujo y devuelve el mensaje de rechazo -sin que el mensaje llegue nunca al LLM principal.
4. Si el resultado es falso (mensaje benigno), el mensaje sigue su curso normal hacia el LLM principal, que genera la respuesta.

Por qué importa la verificación

La acción `check_dangerous_tool_request` imprime evidencia de que se ejecutó de verdad, no que "la respuesta sonó como un rechazo". Con **FakeListLLM** además hay una garantía estructural extra: como el LLM de prueba siempre devuelve la misma respuesta fija, si esa respuesta apareciera para el

mensaje peligroso sería prueba inequívoca de que el rail no bloqueó -no hay forma de que un LLM "alucine" un bloqueo que nunca pasó, porque el LLM ni siquiera decide el contenido de la respuesta de rechazo (viene fijo del Colang). Ojo con las reglas basadas en keywords: son frágiles ante variaciones morfológicas simples (por ejemplo, "elimina" vs. "eliminar" pueden no matchear el mismo patrón).

Resultado esperado

El mensaje benigno pasa y devuelve la respuesta del LLM (de prueba o real); el mensaje peligroso es bloqueado con el texto de rechazo definido en la política, sin que el LLM principal llegue a procesarlo. Comparado con el Lab 3.1: acá el bloqueo ocurre antes de que exista siquiera una decisión de tool a autorizar.